dbt Implementation Plan: Integrating dbt into the Noise-Aware Indian Bird Sound Classification Pipeline

Project: Integrating dbt (data build tool) into the Noise-Aware Indian Bird Sound Classification Pipeline
Team: Samarth D Kolur (ENG23CT0019), Manoj C (ENG23CT0031), Mithun Krishnan (ENG23CT0034), Yashaswini R (ENG23CT0044)
Guide: Prof. Tekumudi Divya, Department of CST, Dayananda Sagar University
Academic Year: 2025–26

Executive Summary

This document is a complete, step-by-step implementation plan for building the dbt (data build tool) project described in the submitted report. The project integrates dbt into the Noise-Aware Indian Bird Sound Classification Pipeline, replacing raw Python/pandas data transformation stages with seven structured, testable, and version-controlled SQL models targeting PostgreSQL as the analytical backend. Since 0% of the implementation is currently complete, this plan covers every action from environment setup through final pipeline validation — aligned precisely with the report's objectives, methodology, architecture, and code listings.

The plan is organised into five sequential phases:

1. Environment Setup and Prerequisites
2. PostgreSQL Database and Schema Setup
3. dbt Project Initialisation and Configuration
4. Model Implementation (all 7 SQL models)
5. Testing, Documentation, and Pipeline Validation

Phase 1: Environment Setup and Prerequisites

1.1 Tools to Install

Before any dbt work begins, every team member must install the following tools on their local machine. This aligns exactly with Table 4.3 (Tools and Technologies) in the report.

Tool / Library	Version	Install Command	Role
Python	3.11+	python.org	Runtime for BirdNET, MLP, AE
PostgreSQL	18	postgresql.org	Analytical warehouse backend

Tool / Library	Version	Install Command	Role
pgAdmin 4	Latest	pgadmin.org	PostgreSQL GUI management
dbt Core	1.7.x	`pip install dbt-core==1.7.0`	SQL model execution, DAG, testing
dbt-postgres adapter	1.7+	`pip install dbt-postgres==1.7.0`	Connects dbt to PostgreSQL
Git	Latest	git-scm.com	Version control, one file per model

Step-by-step install sequence:

```
# Step 1: Create and activate a Python virtual environment
python -m venv dbt_env
source dbt_env/bin/activate        # Linux/macOS
dbt_env\Scripts\activate           # Windows

# Step 2: Install dbt with PostgreSQL adapter
pip install dbt-core==1.7.0 dbt-postgres==1.7.0

# Step 3: Verify installation
dbt --version
# Expected output: dbt Core: 1.7.x   dbt-postgres: 1.7.x
```

1.2 PostgreSQL Setup

After installing PostgreSQL 18, start the PostgreSQL service and confirm it is running:

```
# Linux (systemd)
sudo systemctl start postgresql
sudo systemctl status postgresql

# Windows: Open pgAdmin 4 and verify server is running
```

Phase 2: PostgreSQL Database and Schema Setup

This phase creates the exact database, schema, and source table referenced throughout the report (Section 4.2).

2.1 Create the BIRDNET Database and bird_data Schema

Open pgAdmin 4 or the `psql` CLI and run the following SQL. This matches Listing 4.1 in the report exactly.

```
-- Run in psql or pgAdmin Query Tool
CREATE DATABASE BIRDNET;
\c BIRDNET;
```

```
CREATE SCHEMA bird_data;

CREATE TABLE bird_data.bird_features (
    id            SERIAL PRIMARY KEY,
    bird_species  VARCHAR(255),
    confidence_score FLOAT,
    duration      FLOAT,
    sample_rate   INT,
    audio_path    TEXT
);
```

2.2 Load Sample Data

Since the iBC53 corpus (53 Indian species, 10,044 three-second segments) is used in the report, populate the table with representative test records for development and model validation. Use the following Python script to insert synthetic data matching the corpus statistics:

```
# load_sample_data.py
import psycopg2
import random

SPECIES = [
    "Asian-Koel", "Indian-Robin", "Common-Myna", "Jungle-Babbler",
    "Red-Vented-Bulbul", "Purple-Sunbird", "White-Throated-Kingfisher",
    "Coppersmith-Barbet", "Black-Drongo", "Indian-Peafowl"
]

conn = psycopg2.connect(
    dbname="BIRDNET", user="postgres",
    password="mypassword1234", host="localhost", port=5432
)
cur = conn.cursor()

for i in range(500):
    cur.execute(
        """
        INSERT INTO bird_data.bird_features
            (bird_species, confidence_score, duration, sample_rate, audio_path)
        VALUES (%s, %s, %s, %s, %s)
        """,
        (
            random.choice(SPECIES),
            round(random.uniform(0.3, 0.99), 4),
            round(random.uniform(0.5, 3.0), 3),
            22050,
            f"/data/iBC53/segment_{i:05d}.wav"
        )
    )

conn.commit()
cur.close()
```

```
conn.close()
print("Sample data loaded: 500 rows inserted into bird_data.bird_features")
```

Run with:

```
pip install psycopg2-binary
python load_sample_data.py
```

Phase 3: dbt Project Initialisation and Configuration

3.1 Initialise the dbt Project

```
# Navigate to your working directory
cd ~/projects

# Initialise the dbt project — this creates the folder scaffold
dbt init bird_project
```

When prompted, select `postgres` as the database type. This creates the directory structure shown in Listing 3.1 of the report. Navigate into the project:

```
cd bird_project
```

3.2 Configure profiles.yml

The `profiles.yml` file connects dbt to the PostgreSQL warehouse. This file lives in `~/.dbt/profiles.yml` (outside the project folder for security). Create or edit it to match **Listing 4.2** from the report exactly:

```
# ~/.dbt/profiles.yml
bird_project:
  outputs:
    dev:
      type: postgres
      host: localhost
      user: postgres
      password: mypassword1234
      port: 5432
      dbname: BIRDNET
      schema: public
      threads: 1
  target: dev
```

> **Important:** Replace `mypassword1234` with your actual PostgreSQL password. Never commit `profiles.yml` to Git — add it to `.gitignore`.

3.3 Configure dbt_project.yml

Edit the `dbt_project.yml` in the project root to define model materialisations by layer. All seven models are views in development, as documented in Section 3.5 of the report:

```
# dbt_project.yml
name: 'bird_project'
version: '1.0.0'
config-version: 2

profile: 'bird_project'

model-paths: ["models"]
test-paths: ["tests"]
seed-paths: ["seeds"]
docs-paths: ["docs"]

models:
  bird_project:
    staging:
      +materialized: view
    cleaning:
      +materialized: view
    mart:
      +materialized: view
```

3.4 Set Up the Folder Structure

Create the exact directory layout shown in Listing 3.1 of the report:

```
mkdir -p models/staging
mkdir -p models/cleaning
mkdir -p models/mart
```

Your final folder layout should be:

```
bird_project/
├── models/
│   ├── staging/
│   │   ├── stg_bird_features.sql
│   │   └── sources.yml
│   ├── cleaning/
│   │   ├── clean_bird_features.sql
│   │   └── schema.yml
│   └── mart/
│       ├── bird_distribution.sql
│       ├── bird_stats.sql
│       ├── top_birds.sql
│       ├── high_pitch_birds.sql
│       ├── long_duration_birds.sql
│       └── schema.yml
├── dbt_project.yml
```

```
├── profiles.yml  (kept in ~/.dbt/ outside project)
└── README.md
```

3.5 Verify Connection

Before writing any models, confirm dbt can reach the PostgreSQL database:

```
dbt debug
# Expected: All checks passed
```

Phase 4: Model Implementation (All 7 SQL Models)

This phase implements all seven dbt SQL models in the exact order of the dependency DAG described in Section 3.6 of the report:

```
birddata.bird_features  →  stg_bird_features  →  clean_bird_features
                                                          ↓
                                          bird_distribution  →  top_birds
                                          bird_stats
                                          high_pitch_birds
                                          long_duration_birds
```

4.1 Staging Layer — Model 1: stg_bird_features

File: `models/staging/stg_bird_features.sql`

This is the entry point of the dbt DAG. It reads from the raw PostgreSQL source table using the `source()` macro, selects only the required columns, casts to correct types, and removes null confidence records (Listing 4.4 in the report).

```
-- models/staging/stg_bird_features.sql
-- Selects required columns, casts to correct types, removes null
-- confidence records.
-- Materialised as a view: zero storage cost, full lineage tracking.

SELECT
    id,
    bird_species,
    confidence_score,
    duration,
    sample_rate
FROM {{ source('bird_data', 'bird_features') }}
WHERE confidence_score IS NOT NULL
```

File: `models/staging/sources.yml`

This declares the raw PostgreSQL table as a dbt source, enabling the `source()` macro to function (Listing 4.3 in the report):

```
version: 2

sources:
  - name: bird_data
    database: BIRDNET
    schema: public
    tables:
      - name: bird_features
        description: "Raw bird audio features extracted from iBC53 corpus"
```

4.2 Cleaning Layer — Model 2: clean_bird_features

File: `models/cleaning/clean_bird_features.sql`

This model applies domain-specific quality constraints. It references the staging model via the `ref()` macro, creating the second link in the lineage DAG. It enforces that duration > 0 (valid audio segment), bird_species IS NOT NULL (labelled record), and confidence_score > 0 (non-zero model confidence). This is Listing 4.5 from the report.

```
-- models/cleaning/clean_bird_features.sql
-- Applies domain-specific quality constraints:
-- duration > 0 (valid audio segment)
-- bird_species NOT NULL (labelled record)
-- confidence_score > 0 (non-zero model confidence)

SELECT
    id,
    bird_species,
    confidence_score,
    duration,
    sample_rate
FROM {{ ref('stg_bird_features') }}
WHERE
    duration > 0
    AND bird_species IS NOT NULL
    AND confidence_score > 0
```

4.3 Analytical Mart Layer — Models 3–7

All five mart models reference either `stg_bird_features` or `clean_bird_features` using the `ref()` macro, forming the final layer of the Medallion architecture (Gold layer) as discussed in Section 2.1.8 of the report.

Model 3 — bird_distribution.sql (Species Distribution)

File: `models/mart/bird_distribution.sql`

Computes total sample count per species and ranks by frequency. This enables understanding of class imbalance across the 53 Indian species in iBC53 (Listing 4.6).

```
-- models/mart/bird_distribution.sql

SELECT
    bird_species,
    COUNT(*) AS total_samples
FROM {{ ref('stg_bird_features') }}
GROUP BY bird_species
ORDER BY total_samples DESC
```

Model 4 — bird_stats.sql (Statistical Feature Summary)

File: `models/mart/bird_stats.sql`

Computes aggregate statistics across all audio feature records — average confidence score, maximum duration, and minimum duration — key metrics for dataset health monitoring (Listing 4.7).

```
-- models/mart/bird_stats.sql

SELECT
    AVG(confidence_score) AS avg_confidence,
    MAX(duration)         AS max_duration,
    MIN(duration)         AS min_duration
FROM {{ ref('stg_bird_features') }}
```

Model 5 — top_birds.sql (Top Species Ranking)

File: `models/mart/top_birds.sql`

Extracts the top 10 bird species by sample count from `bird_distribution` — a direct `ref()` to another mart model, demonstrating multi-level DAG chaining (Listing 4.8).

```
-- models/mart/top_birds.sql

SELECT *
FROM {{ ref('bird_distribution') }}
LIMIT 10
```

Model 6 — high_pitch_birds.sql (High-Pitch Species Filtering)

File: `models/mart/high_pitch_birds.sql`

Filters cleaned feature data for species whose average duration falls below 1.5 seconds — a proxy for high-pitch, short-burst vocalisations common in Indian forest soundscapes (Listing 4.9).

```
-- models/mart/high_pitch_birds.sql

SELECT
```

```
    bird_species,
    AVG(confidence_score) AS avg_confidence,
    AVG(duration)        AS avg_duration
FROM {{ ref('clean_bird_features') }}
GROUP BY bird_species
HAVING AVG(duration) < 1.5
ORDER BY avg_confidence DESC
```

Model 7 — long_duration_birds.sql (Long-Duration Species Filtering)

File: `models/mart/long_duration_birds.sql`

Filters for species with sustained vocalisations of 2.0 seconds or longer — identifying sustained callers that occupy longer time windows in the three-second segmentation framework (Listing 4.10).

```
-- models/mart/long_duration_birds.sql

SELECT
    bird_species,
    AVG(confidence_score) AS avg_confidence,
    AVG(duration)        AS avg_duration
FROM {{ ref('clean_bird_features') }}
GROUP BY bird_species
HAVING AVG(duration) >= 2.0
ORDER BY avg_duration DESC
```

Phase 5: Tests, Documentation, and Validation

5.1 Schema Tests (schema.yml)

dbt tests are defined in `schema.yml` files co-located with each model directory. These enforce the data-quality assertions listed in Table 4.2 of the report. Create the following two schema files.

File: `models/cleaning/schema.yml`

This file covers both the staging and cleaning models, and the top_birds mart (as shown in Listing 4.11):

```
version: 2

models:
  - name: stg_bird_features
    description: "Staged bird audio features from iBC53 PostgreSQL source"
    columns:
      - name: id
        tests:
          - not_null
          - unique
```

```
      - name: bird_species
        tests:
          - not_null
      - name: duration
        tests:
          - not_null
      - name: confidence_score
        tests:
          - not_null

  - name: clean_bird_features
    description: "Quality-filtered bird audio features"
    columns:
      - name: bird_species
        tests:
          - not_null
      - name: confidence_score
        tests:
          - not_null

  - name: top_birds
    columns:
      - name: bird_species
        tests:
          - not_null
```

File: `models/mart/schema.yml`

Covers remaining mart models:

```
version: 2

models:
  - name: bird_distribution
    description: "Sample count per species, ordered by frequency"
    columns:
      - name: bird_species
        tests:
          - not_null

  - name: bird_stats
    description: "Aggregate statistics: avg confidence, max/min duration"
    columns:
      - name: avg_confidence
        tests:
          - not_null

  - name: high_pitch_birds
    description: "Species with avg duration < 1.5s (short-burst vocalisers)"
    columns:
      - name: bird_species
        tests:
          - not_null

  - name: long_duration_birds
```

```
    description: "Species with avg duration >= 2.0s (sustained callers)"
    columns:
      - name: bird_species
        tests:
          - not_null
```

5.2 Running the Full Pipeline

Execute the following commands in order. These reproduce exactly the terminal outputs shown in Figures 5.1 and 5.2 of the report.

```
# Step 1: Navigate into the project
cd ~/projects/bird_project

# Step 2: Run all 7 models
dbt run

# Expected terminal output:
# Running with dbt=1.7.x
# Found 7 models, 9 tests, 0 snapshots, 0 analyses, 1 source
#
# Concurrency: 1 threads (target='dev')
#
# 1 of 7 START sql view model public.stg_bird_features ..... [RUN]
# 1 of 7 OK created sql view model public.stg_bird_features ........ [CREATE VIEW ir
# 2 of 7 START sql view model public.clean_bird_features ..... [RUN]
# 2 of 7 OK created sql view model public.clean_bird_features ...... [CREATE VIEW ir
# 3 of 7 START sql view model public.bird_distribution ..... [RUN]
# 3 of 7 OK created sql view model public.bird_distribution ........ [CREATE VIEW i
# 4 of 7 START sql view model public.bird_stats ..... [RUN]
# 4 of 7 OK created sql view model public.bird_stats ..... [CREATE VIEW i
# 5 of 7 START sql view model public.top_birds ..... [RUN]
# 5 of 7 OK created sql view model public.top_birds ..... [CREATE VIEW i
# 6 of 7 START sql view model public.high_pitch_birds ..... [RUN]
# 6 of 7 OK created sql view model public.high_pitch_birds ..... [CREATE VIEW ir
# 7 of 7 START sql view model public.long_duration_birds ..... [RUN]
# 7 of 7 OK created sql view model public.long_duration_birds ...... [CREATE VIEW ir
#
# Finished running 7 view models in ~1.39s.
# Completed successfully. Done. PASS=7 WARN=0 ERROR=0 SKIP=0 TOTAL=7

# Step 3: Run all schema tests
dbt test

# Expected output: 7 tests passed (not_null + unique on all key columns)
# PASS=7 WARN=0 ERROR=0 SKIP=0 TOTAL=7
```

5.3 Running Individual Models

To run or test a single model in isolation (demonstrating the modularity benefit from Section 6.1):

```
# Run only the bird_distribution model
dbt run --select bird_distribution

# Run only the cleaning layer
dbt run --select clean_bird_features

# Test only the staging model
dbt test --select stg_bird_features
```

5.4 Generating Documentation

dbt auto-generates an interactive web catalog from the YAML descriptions. Run the following two commands (mentioned in Section 3.2 of the report):

```
# Generate the documentation artifacts
dbt docs generate

# Serve the catalog locally (opens in browser at http://localhost:8080)
dbt docs serve
```

The catalog displays every model, its compiled SQL, column descriptions, test results, and the lineage DAG graph — matching Figure 3.2 from the report. This documentation is always synchronised with the actual SQL code, unlike manual notebook comments.

Phase 6: Verifying the Dependency DAG

The lineage graph described in Section 3.6 and Figure 3.2 of the report is automatically constructed from the `ref()` and `source()` macros in each model file. After running `dbt docs serve`, navigate to the **Lineage** tab to confirm the following dependency chain is visible:

```
birddata.bird_features
        │
        ▼
stg_bird_features
        │
        ▼
clean_bird_features
    ├── bird_distribution ──► top_birds
    ├── bird_stats
    ├── high_pitch_birds
    └── long_duration_birds
```

This confirms all seven models are correctly wired via DAG resolution as described in the report.

Team Responsibility Allocation

Divide the implementation across the four team members to allow parallel Git-branch development — a key benefit of dbt's one-file-per-model structure (Section 6.1):

Member	USN	Responsibility
Samarth D Kolur	ENG23CT0019	Phase 1 (env setup), Phase 3 (project init + profiles.yml), dbt_project.yml, README
Manoj C	ENG23CT0031	Phase 2 (PostgreSQL setup + sample data loader), sources.yml
Mithun Krishnan	ENG23CT0034	Phase 4 Models 1–2 (stg_bird_features, clean_bird_features), schema.yml tests
Yashaswini R	ENG23CT0044	Phase 4 Models 3–7 (all five mart models), mart/schema.yml, dbt docs

Phased Adoption Path (From Report Section 7.2)

The report recommends a three-phase adoption rather than a complete immediate migration:

Phase	Action	Goal
Phase 1 (Now)	Set up dbt alongside existing Python notebooks. Run all 7 SQL models and verify outputs match Python-computed equivalents	Validate SQL–Python numerical equivalence without disrupting existing pipeline
Phase 2 (After verification)	Replace Python feature aggregation and analytics code with direct PostgreSQL reads from dbt mart models. Run all 7 dbt tests on every CI/CD execution	Fully remove notebook-based transformation code
Phase 3 (Production)	Enable dbt incremental materialisation so new BirdNET-extracted audio feature batches are processed without rebuilding entire historical feature store	Production-grade PAM deployment readiness

Quick-Reference Checklist

Use this checklist to track progress toward a fully working implementation:

- [ ] PostgreSQL 18 installed and running locally
- [ ] `dbt-core==1.7.0` and `dbt-postgres==1.7.0` installed in virtual environment
- [ ] `BIRDNET` database, `bird_data` schema, and `bird_features` table created
- [ ] Sample data loaded into `bird_data.bird_features` (minimum 100 rows)
- [ ] `dbt init bird_project` executed successfully
- [ ] `~/.dbt/profiles.yml` configured for PostgreSQL connection
- [ ] `dbt debug` returns all-pass
- [ ] `models/staging/sources.yml` created
- [ ] `models/staging/stg_bird_features.sql` created

- [ ] `models/cleaning/clean_bird_features.sql` created
- [ ] `models/cleaning/schema.yml` with tests created
- [ ] `models/mart/bird_distribution.sql` created
- [ ] `models/mart/bird_stats.sql` created
- [ ] `models/mart/top_birds.sql` created
- [ ] `models/mart/high_pitch_birds.sql` created
- [ ] `models/mart/long_duration_birds.sql` created
- [ ] `models/mart/schema.yml` with tests created
- [ ] `dbt run` completes with PASS=7
- [ ] `dbt test` completes with PASS=7 (all data-quality assertions pass)
- [ ] `dbt docs generate && dbt docs serve` shows DAG lineage graph

Troubleshooting Common Issues

Error	Likely Cause	Fix
`dbt debug` fails: "Connection refused"	PostgreSQL not running	`sudo systemctl start postgresql`
`Could not connect to server`	Wrong port or password in profiles.yml	Verify `port: 5432` and correct password
`Relation "bird_data.bird_features" does not exist`	Table not created or wrong schema	Re-run Phase 2 SQL setup in pgAdmin
`Source 'bird_data' not found`	sources.yml missing or wrong path	Ensure `models/staging/sources.yml` exists with correct schema name
`ref() function: model not found`	SQL file missing or wrong model name	Check filename matches `ref('model_name')` exactly
`dbt run` creates 0 models	Wrong `model-paths` in dbt_project.yml	Verify `model-paths: ["models"]` is set